

Access Control Lists (ACLs)

Scenario

You are a system administrator managing a shared project folder for a development team. The project folder is shared among three team members: Alice, Bob, and Charlie.

- Alice: Needs full read, write, and execute permissions for the folder.
- Bob: Can only read files but should not modify or delete them.
- Charlie: Needs both read and write permissions but should not delete files.

configure the folder's permissions using Access Control Lists (ACLs) to meet these requirements.

Exercise 1: Create the Project Folder and Test Files

Step 1: Created the Users First because they don't exist

Commands Used:

```
sudo adduser alice
```

```
sudo adduser bob
```

```
sudo adduser charlie
```

Step 2: Created the Group

```
sudo groupadd developers
```

Step 3: Add Users to the Group

```
sudo usermod -aG developers alice
```

```
sudo usermod -aG developers bob
```

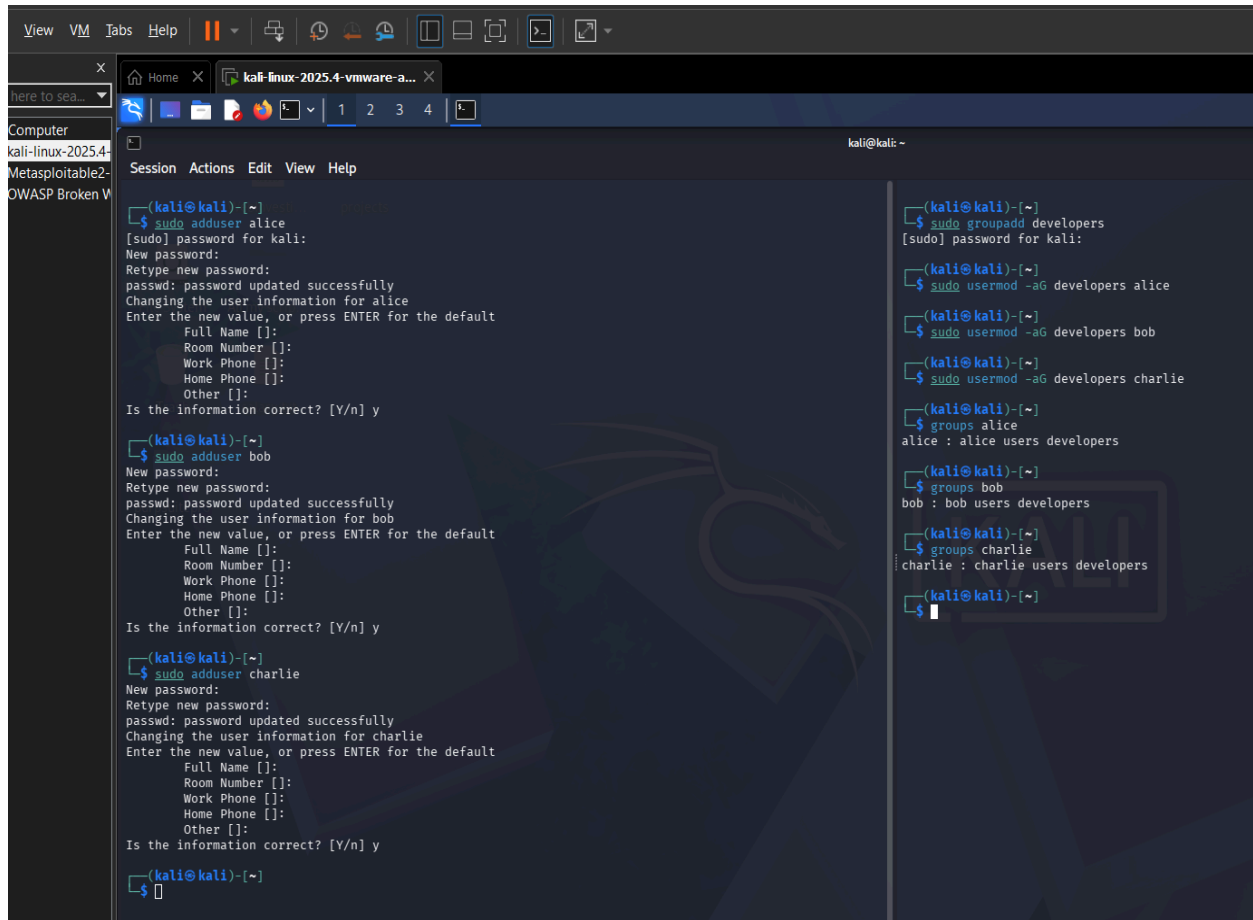
`sudo usermod -aG developers charlie`

Step 4: Verified if successfully created

groups alice

groups bob

groups charlie



```
(kali@kali)-[~]
└─$ sudo groupadd developers
[sudo] password for kali:
└─$ sudo usermod -aG developers alice
└─$ sudo usermod -aG developers bob
└─$ sudo usermod -aG developers charlie
└─$ groups alice
alice : alice users developers
└─$ groups bob
bob : bob users developers
└─$ groups charlie
charlie : charlie users developers
└─$
```

Successfully created users.Group and added users to the group.

Step 5: Create the project folder

Commands Used:

`sudo mkdir /projects/team_project`

To create test files in the folder:

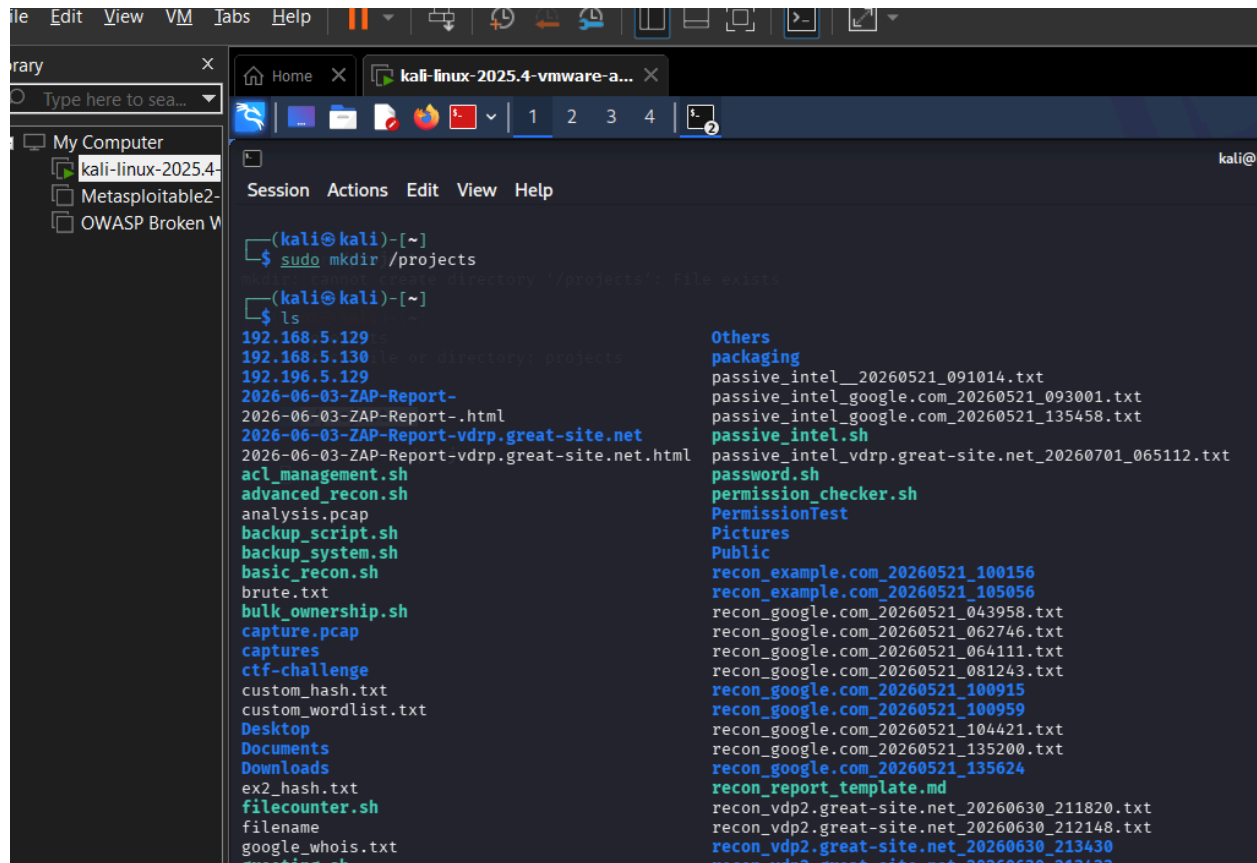
`sudo touch /projects/team_project/{file1.txt,file2.txt}`

Change ownership of the folder to a generic group (e.g., developers)

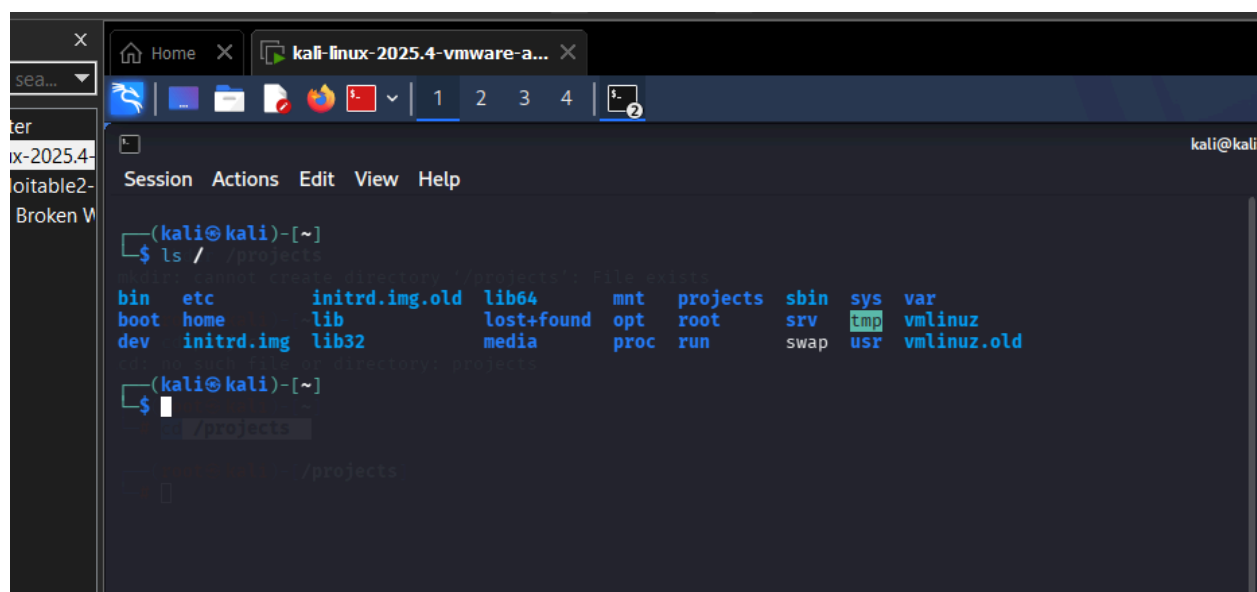
`sudo chown -R root:developers /projects/team_project`

Set default permissions to allow group access:

`sudo chmod 770 /projects/team_project`



```
(kali@kali)-[~]
$ sudo mkdir /projects
mkdir: cannot create directory '/projects': File exists
(kali@kali)-[~]
$ ls
192.168.5.129
192.168.5.130
192.196.5.129
2026-06-03-ZAP-Report-
2026-06-03-ZAP-Report-vdrp.great-site.net
2026-06-03-ZAP-Report-vdrp.great-site.net.html
acl_management.sh
advanced_recon.sh
analysis.pcap
backup_script.sh
backup_system.sh
basic_recon.sh
brute.txt
bulk_ownership.sh
capture.pcap
captures
ctf-challenge
custom_hash.txt
custom_wordlist.txt
Desktop
Documents
Downloads
ex2_hash.txt
filecounter.sh
filename
google_whois.txt
greeting.sh
Others
packaging
passive_intel__20260521_091014.txt
passive_intel_google.com_20260521_093001.txt
passive_intel_google.com_20260521_135458.txt
passive_intel.sh
passive_intel_vdrp.great-site.net_20260701_065112.txt
password.sh
permission_checker.sh
PermissionTest
Pictures
Public
recon_example.com_20260521_100156
recon_example.com_20260521_105056
recon_google.com_20260521_043958.txt
recon_google.com_20260521_062746.txt
recon_google.com_20260521_064111.txt
recon_google.com_20260521_081243.txt
recon_google.com_20260521_100915
recon_google.com_20260521_100959
recon_google.com_20260521_104421.txt
recon_google.com_20260521_135200.txt
recon_google.com_20260521_135624
recon_report_template.md
recon_vdp2.great-site.net_20260630_211820.txt
recon_vdp2.great-site.net_20260630_212148.txt
recon_vdp2.great-site.net_20260630_213430
recon_vdp2.great-site.net_20260630_213433
```



```
(kali@kali)-[~]
$ ls /projects
ls: cannot create directory '/projects': File exists
bin    etc      initrd.img.old  lib64      mnt    projects  sbin  sys  var
boot   home     lib             lost+found  opt    root      srv   tmp  vmlinuz
dev     initrd.img lib32           media      proc   run       swap  usr  vmlinuz.old
```

Successfully created the /projects directory using:

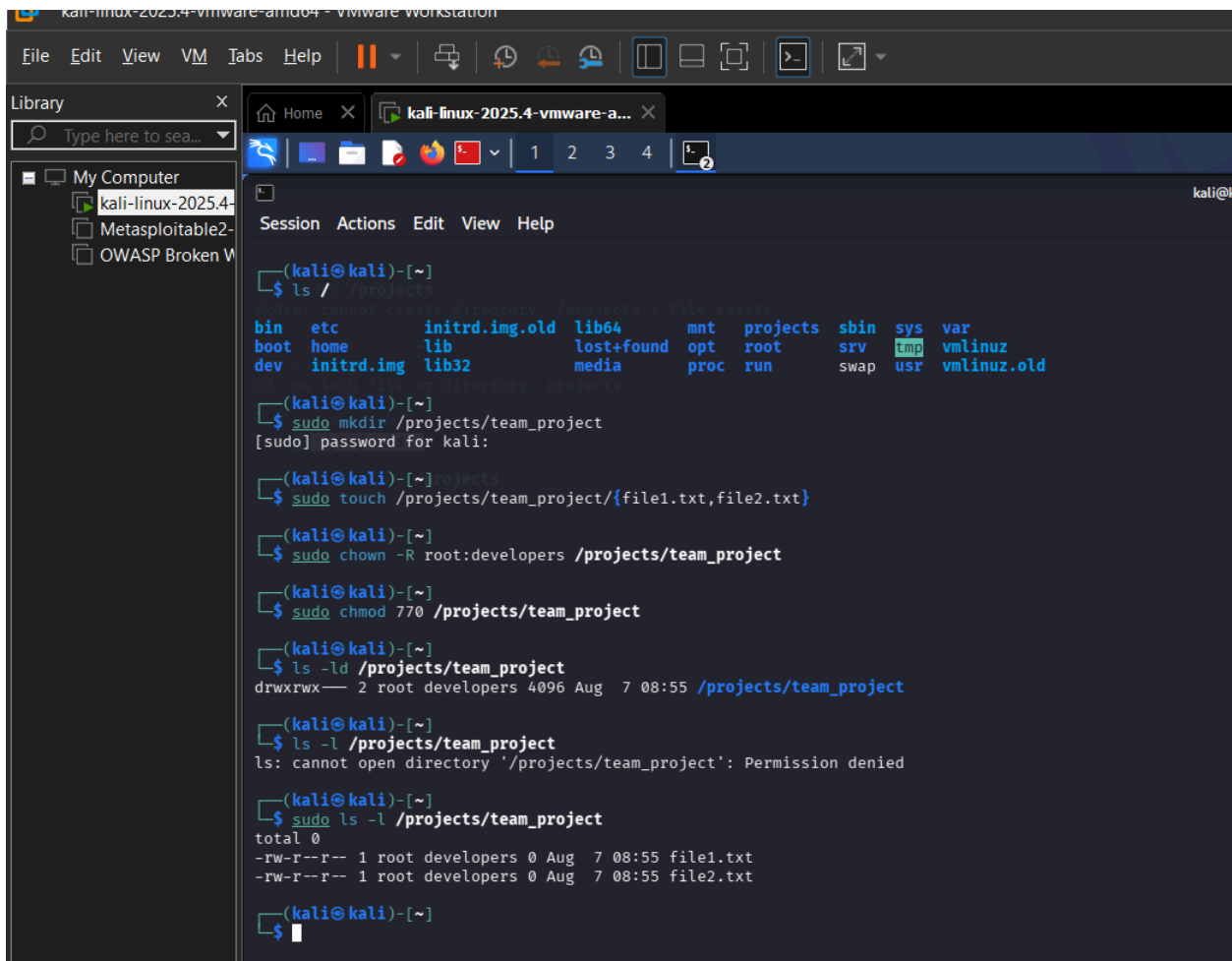
`sudo mkdir /projects`

this placed it in the root of the filesystem. Initially, I couldn't find it with the `ls` command because I was listing my current directory instead of the root directory.

I then used

`ls /`

and confirmed that the projects directory had been created successfully.



```
kali@kali:~$ ls /
bin      etc      initrd.img.old  lib64      mnt      projects  sbin  sys  var
boot    home     lib             lost+found  opt      root      srv   tmp  vmlinuz
dev      initrd.img  lib32          media      proc     run       swap  usr  vmlinuz.old

(kali@kali)-[~]
$ sudo mkdir /projects/team_project
[sudo] password for kali:
(kali@kali)-[~]
$ sudo touch /projects/team_project/{file1.txt,file2.txt}
(kali@kali)-[~]
$ sudo chown -R root:developers /projects/team_project
(kali@kali)-[~]
$ sudo chmod 770 /projects/team_project
(kali@kali)-[~]
$ ls -ld /projects/team_project
drwxrwx--- 2 root developers 4096 Aug  7 08:55 /projects/team_project
(kali@kali)-[~]
$ ls -l /projects/team_project
ls: cannot open directory '/projects/team_project': Permission denied
(kali@kali)-[~]
$ sudo ls -l /projects/team_project
total 0
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file1.txt
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file2.txt
(kali@kali)-[~]
$
```

The team_project directory and the test files were created successfully, The permissions and ownership were applied correctly, although I got a "Permission denied" message when trying to access the directory, I had to use (sudo) because administrative privileges were required, which confirmed that the directory, permissions, ownership, and files were all set up successfully.

Exercise 2: Configure ACLs for Each User

Commands Used:

1. Grant Alice Full Permissions (Read, Write, Execute):

```
sudo setfacl -m u:alice:rwX /projects/team_project
```

2. Grant Bob Read-Only Permissions:

```
sudo setfacl -m u:bob:rx /projects/team_project
```

3. Grant Charlie Read and Write Permissions Without Deletion:

- To prevent deletion, set the sticky bit and assign specific write permissions:

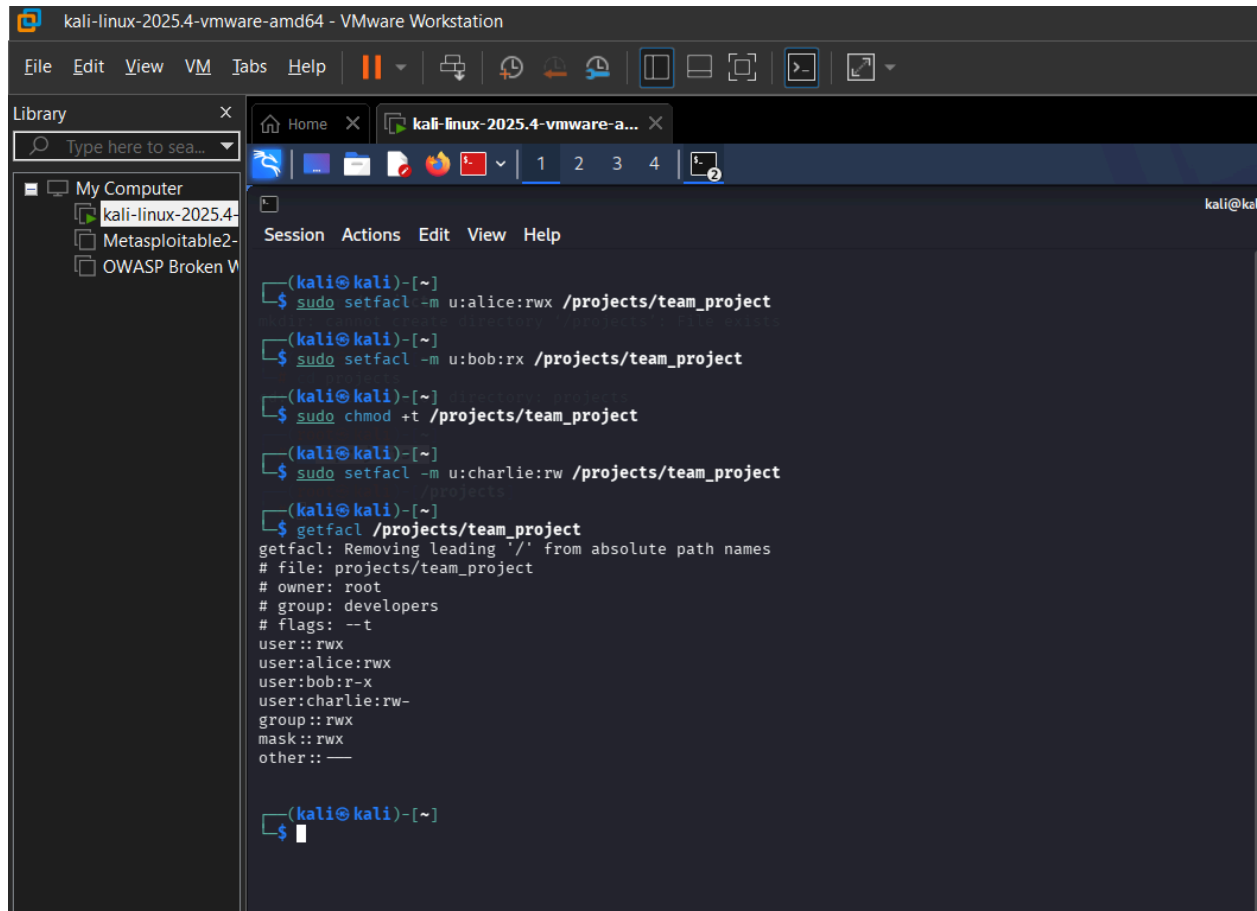
```
sudo chmod +t /projects/team_project sudo setfacl -m u:charlie:rw /projects/team_project
```

4. Verify the ACL Settings:

- Check the ACLs for the directory:

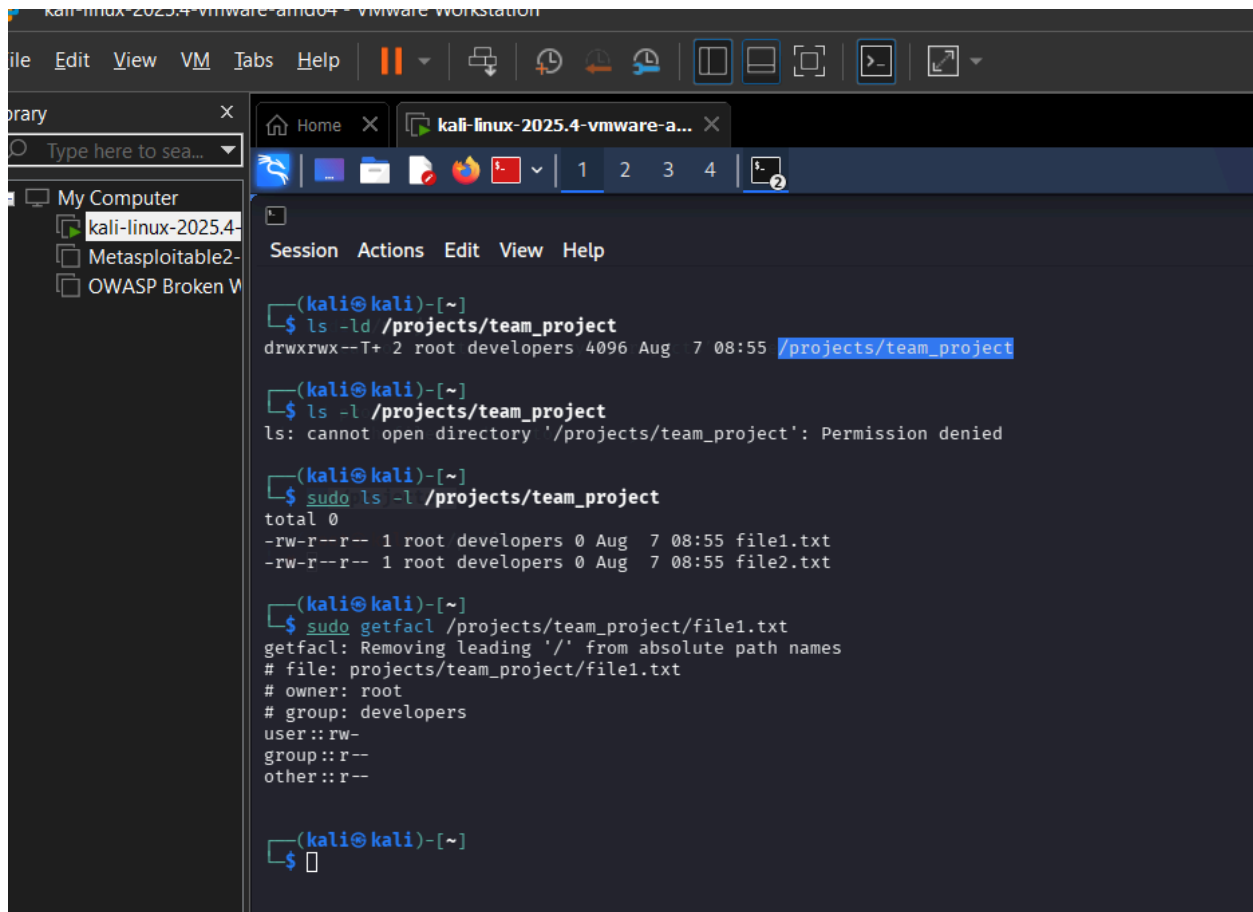
5.

```
getfacl /projects/team_project
```



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken V
kali@kali:~$ sudo setfacl -m u:alice:rwX /projects/team_project
kali@kali:~$ sudo setfacl -m u:bob:rx /projects/team_project
kali@kali:~$ sudo chmod +t /projects/team_project
kali@kali:~$ sudo setfacl -m u:charlie:rw /projects/team_project
kali@kali:~$ getfacl /projects/team_project
getfacl: Removing leading '/' from absolute path names
# file: projects/team_project
# owner: root
# group: developers
# flags: --t
user::rwX
user:alice:rwX
user:bob:r-x
user:charlie:rw-
group::rwX
mask::rwX
other::---
```

The ACLs were successfully configured for Alice, Bob, and Charlie according to the required permissions. The sticky bit was added to the shared project directory to help prevent users from deleting files they do not own. The getfacl output confirmed that all ACL settings were applied successfully.



The screenshot shows a terminal window on a Kali Linux system. The user is in the directory `/projects/team_project`. They attempt to list the directory contents with `ls -ld /projects/team_project`, which succeeds, showing permissions `drwxrwx--T+` and ownership `root developers`. Then, they attempt `ls -l /projects/team_project`, which fails with a "Permission denied" error. Finally, they use `sudo ls -l /projects/team_project` to view the contents, showing two files: `file1.txt` and `file2.txt`, both with permissions `-rw-r--r--` and owned by `root`. They also use `sudo getfacl /projects/team_project/file1.txt` to verify the ACL, showing that the file is owned by `root`, in the `developers` group, and has permissions `user::rw-`, `group::r--`, and `other::r--`.

```
(kali@kali)-[~]
$ ls -ld /projects/team_project
drwxrwx--T+ 2 root developers 4096 Aug  7 08:55 /projects/team_project

(kali@kali)-[~]
$ ls -l /projects/team_project
ls: cannot open directory '/projects/team_project': Permission denied

(kali@kali)-[~]
$ sudo ls -l /projects/team_project
total 0
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file1.txt
-rw-r--r-- 1 root developers 0 Aug  7 08:55 file2.txt

(kali@kali)-[~]
$ sudo getfacl /projects/team_project/file1.txt
getfacl: Removing leading '/' from absolute path names
# file: projects/team_project/file1.txt
# owner: root
# group: developers
user::rw-
group::r--
other::r--

(kali@kali)-[~]
$
```

The directory permissions and ownership were verified successfully. I had to use sudo to view the contents because the current user did not have permission to access the directory, confirming that the access restrictions were working as configured.

Exercise 3: Test the Permissions

1. Switch to each user and test their access rights:

- For Alice:

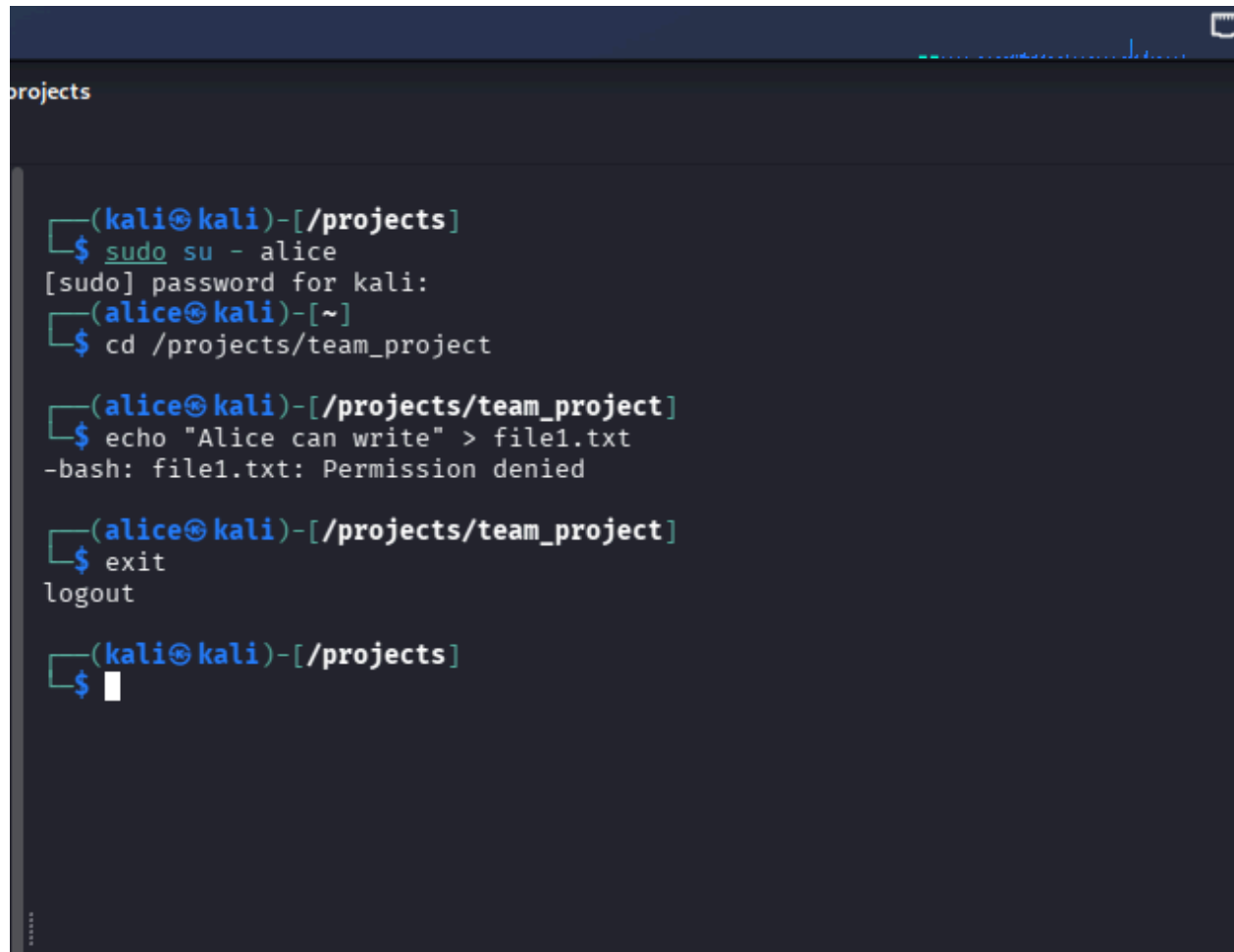
```
sudo su - alice cd /projects/team_project echo "Alice can write" > file1.txt
```

- For Bob:

```
sudo su - bob cd /projects/team_project cat file1.txt echo "Bob can write" >>
file1.txt # This should fail.
```

- For Charlie:

sudo su - charlie cd /projects/team_project echo "Charlie can write" > file2.txt rm file1.txt # This should fail due to the sticky bit.



```
(kali㉿kali)-[/projects]
$ sudo su - alice
[sudo] password for kali:
(alice㉿kali)-[~]
$ cd /projects/team_project

(alice㉿kali)-[/projects/team_project]
$ echo "Alice can write" > file1.txt
-bash: file1.txt: Permission denied

(alice㉿kali)-[/projects/team_project]
$ exit
logout

(kali㉿kali)-[/projects]
$
```

I switched to the Alice account and attempted to write to file1.txt,

Error Message

Received a "**Permission denied**" error. This showed that although the ACLs were configured on the directory, additional permissions were still required for the existing txt file.

What i did to rectify the error

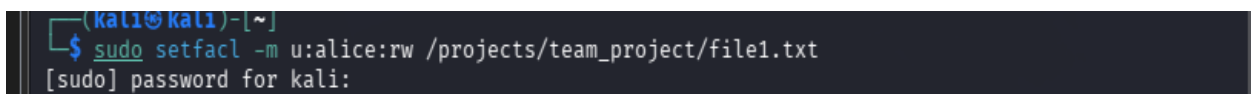
apply ACLs to the txt files as well.

Commands Used:

```
sudo setfacl -m u:alice:rw /projects/team_project/file1.txt  
sudo setfacl -m u:bob:r /projects/team_project/file1.txt  
sudo setfacl -m u:charlie:rw /projects/team_project/file2.txt
```

verified:

```
sudo getfacl /projects/team_project/file1.txt
```



```
(kali㉿kali)-[~]  
$ sudo setfacl -m u:alice:rw /projects/team_project/file1.txt  
[sudo] password for kali:
```

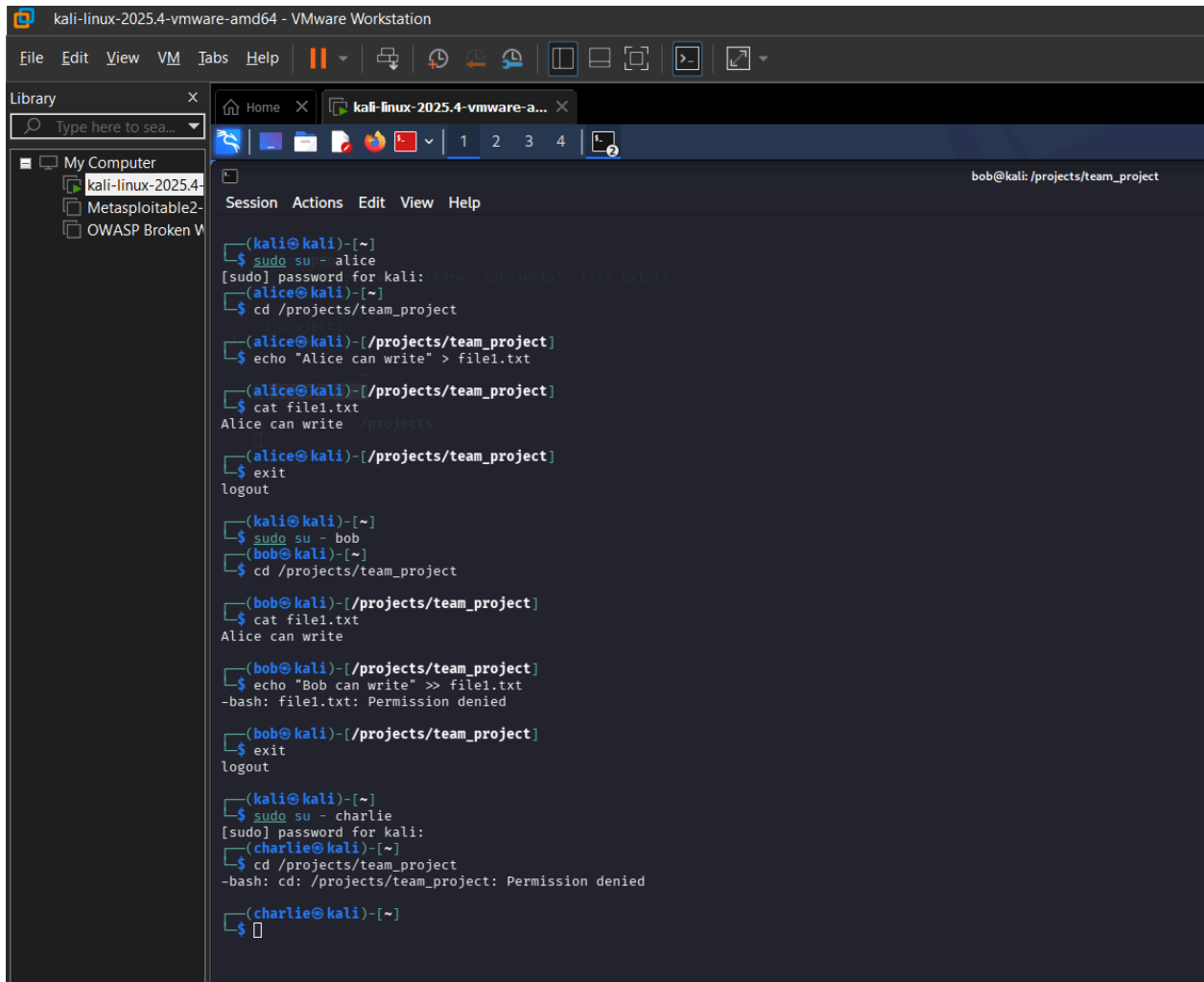
```
(kali㉿kali)-[/projects]
$ sudo setfacl -m u:bob:r /projects/team_project/file1.txt

(kali㉿kali)-[/projects]
$ sudo setfacl -m u:charlie:rw /projects/team_project/file2.txt

(kali㉿kali)-[/projects]
$ sudo getfacl /projects/team_project/file1.txt
getfacl: Removing leading '/' from absolute path names
# file: projects/team_project/file1.txt
# owner: root
# group: developers
user::rw-
user:alice:rw-
user:bob:r--
group::r--
mask::rw-
other::r--

(kali㉿kali)-[/projects]
$
```

Tested for Permissions again



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.4
Metasploitable2
OWASP Broken VM
Home
kali-linux-2025.4-vmware-a...
1 2 3 4
bob@kali: /projects/team_project
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo su - alice
[sudo] password for kali: 
(alice@kali)-[~]
$ cd /projects/team_project
(alice@kali)-[/projects/team_project]
$ echo "Alice can write" > file1.txt
(alice@kali)-[/projects/team_project]
$ cat file1.txt
Alice can write /projects
(alice@kali)-[/projects/team_project]
$ exit
logout
(kali@kali)-[~]
$ sudo su - bob
(bob@kali)-[~]
$ cd /projects/team_project
(bob@kali)-[/projects/team_project]
$ cat file1.txt
Alice can write
(bob@kali)-[/projects/team_project]
$ echo "Bob can write" >> file1.txt
-bash: file1.txt: Permission denied
(bob@kali)-[/projects/team_project]
$ exit
logout
(kali@kali)-[~]
$ sudo su - charlie
[sudo] password for kali: 
(charlie@kali)-[~]
$ cd /projects/team_project
-bash: cd: /projects/team_project: Permission denied
(charlie@kali)-[~]
$
```

Alice was able to access the project directory, write to **file1.txt**, and verify its contents successfully. This confirmed that her read, write, and execute permissions were configured correctly.

The ACLs were successfully applied to **file1.txt**, giving **Alice** read and write access while **Bob** was granted read-only access. The `getfacl` output confirmed that the permissions were configured correctly for each user.

Charlie was unable to access the **team_project** directory because he did not have execute (x) permission on the directory. As a result, Linux correctly denied access, preventing him from reaching the files inside.

Exercise 4: Manage Default ACLs

1. Set default ACLs so new files inherit permissions:

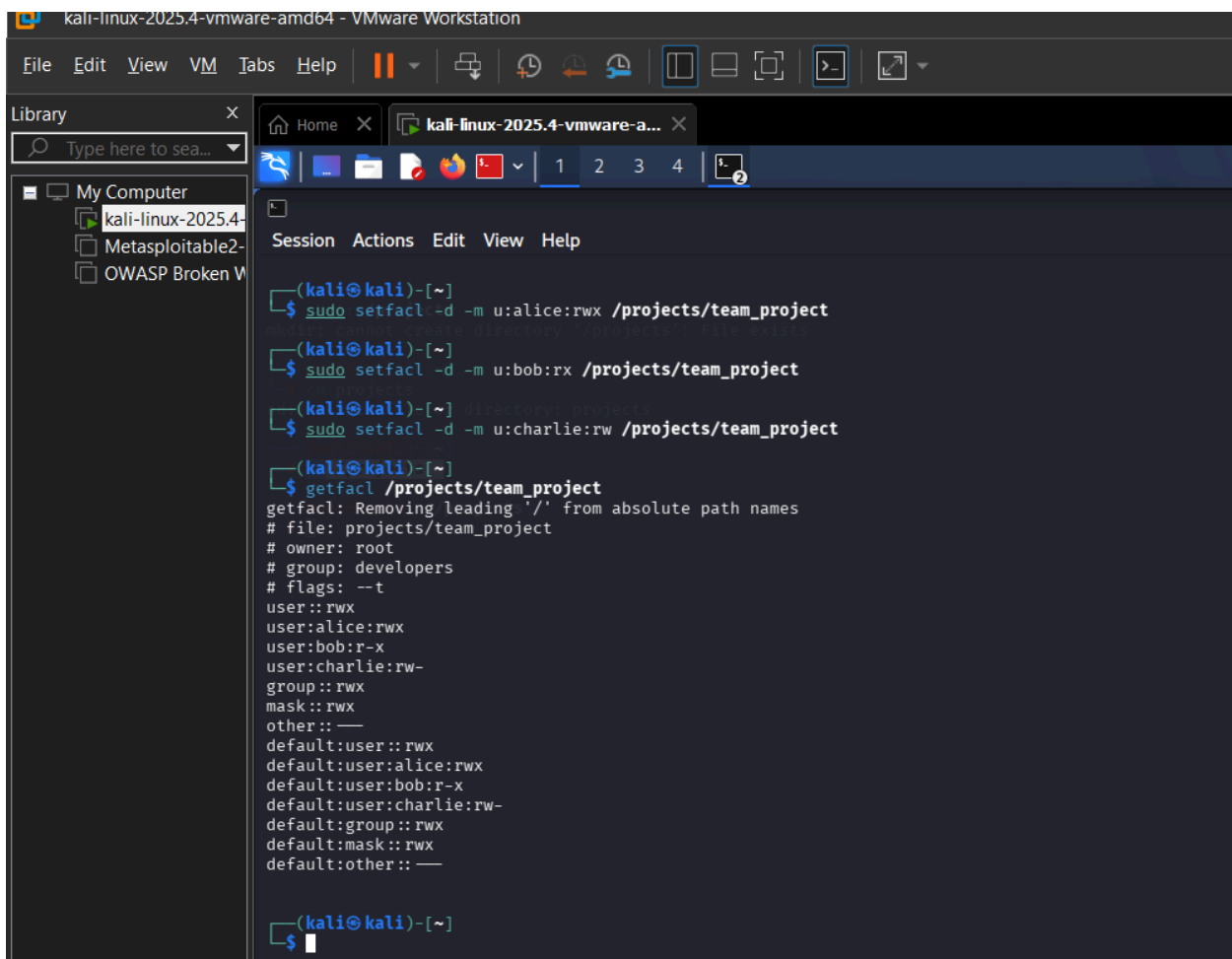
```
sudo setfacl -d -m u:alice:rw /projects/team_project
```

```
sudo setfacl -d -m u:bob:r-x /projects/team_project
```

```
sudo setfacl -d -m u:charlie:rw /projects/team_project
```

2. Verify default ACLs:

```
getfacl /projects/team_project
```



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo setfacl -d -m u:alice:rw /projects/team_project
(kali@kali)-[~]
$ sudo setfacl -d -m u:bob:r-x /projects/team_project
(kali@kali)-[~]
$ sudo setfacl -d -m u:charlie:rw /projects/team_project
(kali@kali)-[~]
$ getfacl /projects/team_project
getfacl: Removing leading '/' from absolute path names
# file: projects/team_project
# owner: root
# group: developers
# flags: --t
user::rw
user:alice:rw
user:bob:r-x
user:charlie:rw-
group::rw
mask::rw
other::---
default:user::rw
default:user:alice:rw
default:user:bob:r-x
default:user:charlie:rw-
default:group::rw
default:mask::rw
default:other::---
```

The default ACLs were successfully configured for Alice, Bob, and Charlie on the team_project directory. The getfacl output confirmed that all new files and directories created inside the project folder will automatically inherit these permissions
